```
import { readFileSync } from 'node:fs';
import { createServer } from 'node:http';
import { timingSafeEqual } from 'node:crypto';
import {
  EVENTO, MAX_CARTONES_POR_COMPRA, PREMIOS_EVENTO, PRECIO_CARTON, PRECIO_ENTRADA,
} from './evento.js';
import { aplicarEsquema, makePool } from './db.js';
import { elegirMp, exigirCuentaDePrueba } from './config.js';
import { firmar, verificarFirma } from './mp.js';
import {
  ErrorNegocio, ajustarCompra, cancelarCompra, cancelarOrden, conciliacion, crear
  guardarPreferencia, liberarVencidas, panel, stock, procesarPago, transferirComp
} from './orders.js';

// La web es un solo HTML. Al servirla desde acá se le avisa que hay servidor (mo
let htmlBase = null;
function paginaWeb(modo, premios = []) {
  htmlBase ??= readFileSync(new URL('../../web/bingo-balmoral.html', import.meta.
  const datos = {
    evento: EVENTO, precioEntrada: PRECIO_ENTRADA, precioCarton: PRECIO_CARTON, t
    maxPorCompra: MAX_CARTONES_POR_COMPRA, premios,
  };
  const json = JSON.stringify(datos).replace(/</g, '\u003c');
  return htmlBase.replace('<body>', `<body><script>window.RIFA_API = ""; window.R
}

const json = (res, status, body) => {
  res.writeHead(status, { 'content-type': 'application/json; charset=utf-8' });
  res.end(JSON.stringify(body));
};

async function leerJson(req, max = 20_000) {
  let data = '';
  for await (const chunk of req) {
    data += chunk;
    if (data.length > max) throw new ErrorNegocio('cuerpo_grande', 'Cuerpo demasi
  }
  return data ? JSON.parse(data) : {};
}

const iguales = (a, b) => {
  const x = Buffer.from(String(a ?? '')); const y = Buffer.from(String(b ?? ''));
  return x.length === y.length && timingSafeEqual(x, y);
};
```

```
// Límite simple por IP para que nadie bloquee los números creando órdenes en mas
function limitador(maxPorMinuto) {
  const hits = new Map();
  return (ip) => {
    const ahora = Date.now();
    const lista = (hits.get(ip) ?? []).filter((t) => ahora - t < 60_000);
    lista.push(ahora); hits.set(ip, lista);
    return lista.length <= maxPorMinuto;
  };
}

/** Arma el servidor. `mp` es la interfaz de Mercado Pago (hoy: simulada). */
export function crearApp({ pool, mp, config = {} }) {
  const {
    baseUrl = 'http://localhost:3000',
    webhookSecret = null,        // si está, el webhook exige firma válida
    panelKey = null,             // si no está, el panel queda deshabilitado
    modo = 'simulado',
    premios = PREMIOS_EVENTO,    // premios que se muestran en la web (evento.js,
    maxOrdenesPorMinuto = 10,
  } = config;
  const permitido = limitador(maxOrdenesPorMinuto);
  const permitidoStock = limitador(120);

  async function webhook(req, res, url) {
    const cuerpo = await leerJson(req).catch(() => ({}));
    const dataId = url.searchParams.get('data.id') ?? cuerpo?.data?.id ?? url.sea
    const tipo = url.searchParams.get('type') ?? url.searchParams.get('topic') ??
    if (webhookSecret) {
      const ok = verificarFirma({
        secreto: webhookSecret, firma: req.headers['x-signature'],
        requestId: req.headers['x-request-id'], dataId,
      });
      if (!ok) return json(res, 401, { error: 'firma_invalida' });
    }
    if (tipo !== 'payment' || !dataId) return json(res, 200, { ignorado: true });
    try {
      const r = await procesarPago(pool, mp, dataId);
      return json(res, 200, r);
    } catch (e) {
      console.error('webhook', e);
      return json(res, 500, { error: 'reintentar' }); // MP vuelve a avisar
    }
  }

  async function crearPago(req, res) {
```

```
    const ip = req.socket.remoteAddress ?? '?';
    if (!permitido(ip)) return json(res, 429, { error: 'demasiados_intentos' });
    const b = await leerJson(req);
    const orden = await crearOrden(pool, {
      // Los cartones se asignan solo al azar: el servidor ignora cualquier pedid
      nombre: b.nombre, whatsapp: b.whatsapp, entradas: b.entradas, sueltos: b.su
    });
    try {
      const pref = await mp.createPreference({
        ref: orden.ref, monto: orden.monto, titulo: `${EVENTO.nombre} ${EVENTO.in
        expiraEn: orden.expiraEn,
      });
      await guardarPreferencia(pool, orden.id, pref.id);
      return json(res, 201, { ref: orden.ref, monto: orden.monto, init_point: pre
    } catch (e) {
      await cancelarOrden(pool, orden.id);
      throw e;
    }
  }

  // Pantalla de pago simulada (solo modo prueba): reemplaza al checkout de Merca
  async function mockCheckout(req, res, url) {
    const ref = url.pathname.split('/')[2];
    const o = await estadoOrden(pool, ref);
    if (!o) return json(res, 404, { error: 'orden_inexistente' });
    if (req.method === 'GET') {
      res.writeHead(200, { 'content-type': 'text/html; charset=utf-8' });
      return res.end(`<!doctype html><meta charset=utf-8><meta name=viewport cont
<title>Pago de prueba</title><body style="font:20px sans-serif;max-width:480px;ma
<p style="background:#f2c200;padding:8px;font-weight:700">MODO PRUEBA · no es Mer
<h2>Pagar $${o.monto.toLocaleString('es-AR')}</h2><p>Orden ${o.ref}</p>
<form method=post><button name=status value=approved style="width:100%;min-height
<button name=status value=rejected style="width:100%;min-height:64px;font:700 20p
    }
    let data = ''; for await (const ch of req) data += ch;
    const status = new URLSearchParams(data).get('status') === 'approved' ? 'appr
    const pid = mp.simularPago({ ref, amount: o.monto, status });
    // Como haría Mercado Pago: avisa al webhook (con firma si está configurada).
    const rid = `req-${pid}`;
    const headers = { 'content-type': 'application/json', 'x-request-id': rid };
    if (webhookSecret) headers['x-signature'] = firmar({ secreto: webhookSecret,
    await fetch(`${baseUrl}/api/mp-webhook?type=payment&data.id=${pid}`, { method
    res.writeHead(303, { location: `/?orden=${ref}${status === 'approved' ? '' :
    return res.end();
  }

  return createServer(async (req, res) => {
```

```
    const url = new URL(req.url, baseUrl);
    try {
      if (req.method === 'GET' && (url.pathname === '/' || url.pathname === '/ind
        res.writeHead(200, { 'content-type': 'text/html; charset=utf-8', 'cache-c
        return res.end(paginaWeb(modo, premios));
      }
      if (req.method === 'GET' && url.pathname === '/api/stock') {
        if (!permitidoStock(req.socket.remoteAddress ?? '?')) return json(res, 42
        return json(res, 200, await stock(pool));
      }
      if (req.method === 'POST' && url.pathname === '/api/crear-pago') return awa
      if (req.method === 'POST' && url.pathname === '/api/mp-webhook') return awa
      if (req.method === 'POST' && (url.pathname === '/api/panel/cancelar' || url
        if (!panelKey || !iguales(req.headers['x-panel-key'], panelKey)) return j
        const b = await leerJson(req);
        return json(res, 200, url.pathname === '/api/panel/cancelar' ? await canc
      }
      if (req.method === 'POST' && (url.pathname === '/api/panel/ajustar' || url.
        if (!panelKey || !iguales(req.headers['x-panel-key'], panelKey)) return j
        const b = await leerJson(req);
        return json(res, 200, url.pathname === '/api/panel/ajustar'
          ? await ajustarCompra(pool, { ref: b.ref, entradas: b.entradas, sueltos
          : await crearCortesia(pool, { nombre: b.nombre, whatsapp: b.whatsapp, e
      }
      if (req.method === 'POST' && url.pathname === '/api/panel/transferir') {
        if (!panelKey || !iguales(req.headers['x-panel-key'], panelKey)) return j
        const b = await leerJson(req);
        return json(res, 200, await transferirCompra(pool, { desde: b.desde, haci
      }
      if (req.method === 'GET' && url.pathname.startsWith('/api/orden/')) {
        const o = await estadoOrden(pool, url.pathname.split('/')[3]);
        return o ? json(res, 200, o) : json(res, 404, { error: 'orden_inexistente
      }
      if (url.pathname.startsWith('/mock-mp/') && modo === 'simulado' && mp.simul
      if (req.method === 'GET' && (url.pathname === '/api/panel' || url.pathname
        if (!panelKey || !iguales(req.headers['x-panel-key'], panelKey)) return j
        return json(res, 200, url.pathname === '/api/panel' ? await panel(pool) :
      }
      return json(res, 404, { error: 'no_encontrado' });
    } catch (e) {
      if (e instanceof ErrorNegocio) return json(res, 400, { error: e.codigo, men
      if (e instanceof SyntaxError) return json(res, 400, { error: 'json_invalido
      console.error(e);
      return json(res, 500, { error: 'error_interno' });
    }
  });
}
```

```
// Arranque: `npm start`. El modo lo decide MP_MODO (ver src/config.js).
if (import.meta.url === `file://${process.argv[1]}`) {
  const puerto = Number(process.env.PORT ?? 3000);
  process.env.PORT = String(puerto);
  const { modo, baseUrl, mp, verificarCuenta, inicioToken } = elegirMp();
  if (verificarCuenta) await exigirCuentaDePrueba(mp, inicioToken);
  const pool = makePool();
  await aplicarEsquema(pool);
  // Defensa: si esta base ya tiene otra cantidad de números (por ejemplo, la de
  const hay = (await pool.query('select count(*)::int n from numeros')).rows[0].n
  if (hay !== TOTAL_CARTONES) {
    console.error(`La base tiene ${hay} cartones y el bingo necesita ${TOTAL_CART
    process.exit(1);
  }
  const app = crearApp({
    pool, mp,
    config: {
      baseUrl, modo, webhookSecret: process.env.MP_WEBHOOK_SECRET || null, panelK
      premios: leerPremios(process.env.PREMIOS).length ? leerPremios(process.env.
    },
  });
  setInterval(() => liberarVencidas(pool).catch(console.error), 60_000).unref();
  app.listen(puerto, () => console.log(`Bingo en ${baseUrl} · modo ${modo.toUpper
}
```